

Файлове в Python. Try..Ексерпт.

Файлове в Python.

Ключовата функция за работа с файлове в Python е `open()`. Тя приема като параметри име на файла и режим в който той е отварян. Режимите за отваряне на файл са:

- 'r' - Read - Това е режимът, в който файл се отваря по подразбиране. Файлът се отваря за четене, като ако същият не съществува се генерира грешка.
- 'a' - Append - Файлът се отваря за добавяне на данни. Ако файлът не съществува се създава.
- 'w' - Write - Файлът се отваря за писане. Ако файлът не съществува се създава.
- 'x' - Create - Файлът се създава, като ако не съществува се генерира грешка.

Файловете могат да бъдат отворени в текстов и двоичен режим:

- 't' - Text mode.
- 'b' - Binary mode.

```
In [1]: my_file = open('newfile.txt', 'w')
my_file.write('Hello, World!\nWhat a wonderful day!')
my_file.close()
```

Горният блок от код е написан за да създадем файл, който съдържа текстова информация на два реда с демонстрационна цел.

```
In [2]: my_file = open('newfile.txt')
print(my_file)

<_io.TextIOWrapper name='newfile.txt' mode='r' encoding='cp1252'>
```

```
In [3]: # Идентично отваряне. Подали сме стойностите по подразбиране за режим на
отваряне.
myfile = open('newfile.txt', 'rt')
```

Както видяхме, ако се опитаме да изведем `my_file` получаваме входно-изходен обект. За да видим какво е съдържанието на файла трябва да използваме метода `read()`.

```
In [4]: my_file = open('newfile.txt')
print(my_file.read())
```

```
Hello, World!
What a wonderful day!
```

Също така можем да посочим каква част от файла да прочетем.

```
In [5]: my_file = open('newfile.txt')
        print(my_file.read(5))
```

Hello

Можем да прочетем и един ред от файла с `.readline()` метода.

```
In [6]: my_file = open('newfile.txt')
        print(my_file.readline())
        print(my_file.readline())
```

Hello, World!

What a wonderful day!

Затваряне на файлове

След като бъдат отворени и работата с тях е приключена, файловете трябва да бъдат затворени. Това става с `.close()` метода.

```
In [7]: my_file = open('newfile.txt')
        print(my_file.read(5))
        my_file.close()
```

Hello

Писане във файлове.

Писането на файлове става с `.write()` метода след като сме отворили файла в подходящ режим.

```
In [8]: # Open file in write mode
        my_file = open('some_file.txt', 'w')
        my_file.write('Hello there!')
        my_file.close()
```

```
In [9]: # Open file in append mode
        my_file = open('some_file.txt', 'a')
        my_file.write('Hello again!')
        my_file.close()
```

```
In [10]: my_file = open('some_file.txt', 'r')
         print(my_file.read())
         my_file.close()
```

Hello there!Hello again!

Try..Except в Python

Пълната форма на Try..Except включва:

- try - кодът в този блок се тества за грешки
- except - в този блок обработваме грешката (какво ще стане при възникнала дадена грешка / даден тип грешки и т.н.)
- else - кодът в този блок се изпълнява ако нямаме грешка
- finally - кодът в този блок се изпълнява независимо от случилото се в try..except блоковете

Exception handling в Python

Когато настъпи някаква грешка (или изключение - exception) без тя да е обработена Python спира изпълнението и генерира изключение. Тези изключения могат да бъдат обработвани в try.

```
In [11]: # Генерира се съобщение, тъй като променлива dog не съществува.
try:
    print(dog)
except:
    print('An exception occurred!')
```

An exception occurred!

Тъй като сме обработили грешката ни цялостното изпълнение на програмата не се прекъсва. Ако не обработим грешката получаваме:

```
In [12]: print(dog)
```

```
-----
NameError                                Traceback (most recent call last)
Cell In [12], line 1
----> 1 print(dog)

NameError: name 'dog' is not defined
```

Множество изключения

Възможно е и да обработваме различни изключения по различен начин т.е. да изпълняваме различен кодов блок при настъпването на различни изключения. При обработването на грешки приключваме с изпълнението на кода в try при първата настъпила грешка. В примера отдолу се опитваме да изведем променлива dog на екрана, която не е дефинирана и след това се опитваме да приложим операция, която не се поддържа от символни низове.

```
In [13]: try:
    print(dog)
    name = 'John'
    name /= 2

except NameError:
    print('Undefined variable!')

except:
    print('General error occurred!')
```

Undefined variable!

Else

В блок else посочваме код, който ще се изпълни когато нямаме настъпили изключения в try.

```
In [14]: try:
          print("Hello")
        except:
          print("Something went wrong")
        else:
          print("Nothing went wrong")
```

```
Hello
Nothing went wrong
```

Finally

Кодът във Finally изпълняваме независимо от това дали са настъпили някакви изключения или не.

```
In [15]: try:
          print(x)
        except:
          print('Something went wrong')
        else:
          print('Nothing went wrong!')
        finally:
          print('The \'try except\' is finished')
```

```
Something went wrong
The 'try except' is finished
```

Raise an exception.

Като програмисти можем да генерираме изключения (throw / raise an exception).

```
In [16]: age = 21.1

if type(age) != int:
    raise TypeError(f'Age variable cannot be of type {type(age)}')
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In [16], line 4
      1 age = 21.1
      3 if type(age) != int:
----> 4     raise TypeError(f'Age variable cannot be of type {type(age)}')

TypeError: Age variable cannot be of type <class 'float'>
```

Самостоятелни задачи.

1. Напишете програма, която чете файл ред по ред и записва информацията в лист.
2. Напишете програма, която чете информация за студенти от файл и ги записва в речник. Създайте функция, която извежда информацията за един / няколко / всички студенти. Създайте функция, която извежда средния успех за дадена група. Структурата на данните следва формата:

<име1>, <фамилия1>, <възраст1>, <пол1>, <среден успех1>, <група1>

<име2>, <фамилия2>, <възраст2>, <пол2>, <среден успех2>, <група2>...

3. Добавете подходящ exception handling на задача 2, подsigуряващ правилни типове данни за отделните полета.
4. Напишете програма, която маха новите редове от даден файл.